Try-Catch, Exceptions and Throw/Throws in Java

Try-Catch

There are two main parts to the Try-Catch; one is the Try block and the other is the Catch block. You can think of the Try-Catch like an if-else statement. It functions very similar to basic if-else.

Syntax is as follows:

```
try                     //this is the try block
{
    //code that may throw an exception

}catch(Exception identifier)  //this is the catch block
{                       //catch needs Exception as parameter type and identifier
    //code
}
```

Try Block:
A try block is the block of code (that contains a set of statements) in which exceptions can occur; it's used to enclose the code that might throw an exception. The try block is always followed by a catch block, which handles the exception that occurs in the associated try block. A try block must be used within the method and it must be followed by a catch block(s).

Catch Block:
The catch block catches and handles the try block exceptions by declaring the type of exception within the parameter and needs an identifier. The catch block includes the code, and it is executed if an exception inside the try block occurs. The catch block is where you handle the exceptions; so, this block must follow the try block. You also need to declare an exception in the catch block (see below for more on exceptions). You are allowed to have more than one catch block (more than one exception) with each try block.

Exceptions

An exception (or exceptional event) is a problem that arises during the execution of a program. When an Exception occurs during the execution of program (application), the normal flow of the program is disrupted, and the program/Application terminates abnormally whereby the whole coding may corrupt (if there was no backup). Therefore, these exceptions must be handled for a smooth run.

There are two general types of Exceptions; Checked and Unchecked.

Checked: are checked at compile-time.
Unchecked: are checked at run-time.

Java Exceptions can be built-in or user-defined, this handout will only cover built-in Exceptions.

Built-in exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations. Below is the list of important built-in exceptions in Java, there are many more built-in exceptions than what is listed below.

1. **ArithmeticException:** It is thrown when an exceptional condition has occurred in an arithmetic operation.
2. **ArrayIndexOutOfBoundsException:** It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.
3. **ClassNotFoundException:** This Exception is raised when we try to access a class whose definition is not found.
4. **FileNotFoundException:** This Exception is raised when a file is not accessible or does not open.
5. **IOException:** It is thrown when an input-output operation failed or interrupted.
6. **NoSuchFieldException:** It is thrown when a class does not contain the field (or variable) specified.
7. **NoSuchMethodException:** It is thrown when accessing a method that is not found.
8. **NullPointerException:** This exception is raised when referring to the members of a null object. Null represents nothing.
9. **NumberFormatException:** This exception is raised when a method could not convert a string into a numeric format.
10. **RuntimeException:** This represents an exception that occurs during runtime.
11. **StringIndexOutOfBoundsException:** It is thrown by String class methods to indicate that an index is either negative or greater than the size of the string.
12. **IllegalArgumentException :** This exception will throw the error or error statement when the method receives an argument which is not accurately fit to the given relation or condition. It comes under the unchecked exception.

Throw/Throws:

Both `throw` and `throws` are concepts of exception handling in Java. The `throws` keyword is used to declare which exceptions can be thrown **from** a method/function, while the `throw` keyword is used to explicitly throw an exception **within** a method/function or block of code.

The `throws` keyword is used in a method/function declaration and declares which exceptions can be thrown from a method and this means that the method/function itself will not handle the exception in a try-catch but could cause program to terminate.

On the other hand, the `throw` keyword is used within a method/function body, or any block of code, and is used to explicitly throw a single exception. The `throw` keyword can be useful for throwing exceptions based on certain conditions within a code block and for throwing custom exceptions.

The Java **throws** keyword is used to declare an exception in the method/function declaration. It tells the programmer that there may occur an exception. So the Try-Catch is not needed, but is recommended. It is better for the programmer to provide the exception handling code so that the normal flow of the program can be maintained.

Exception Handling is mainly used to handle the **checked** exceptions. If there occurs any unchecked exception such as NullPointerException, it is programmers' fault that he is not checking the code before it being used.